

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA EN CIBERSEGURIDAD



Título:

**PROGRAMACIÓN ORIENTADA A OBJETOS EN C++ Y SU APLICACIÓN
EN ESTRUCTURAS DE DATOS Y ALGORITMOS**

Asignatura:

ESTRUCTURA DE DATOS Y ALGORITMOS

Docente:

Mg. CARLOS ABRAHAM RIOS RIVERA

Estudiante:

CONDOR POMA, MIGUEL ANGEL

Tingo María – Perú

2026

RESUMEN

La Programación Orientada a Objetos (POO) es un enfoque muy utilizado en la creación de software moderno. Ayuda a organizar y estructurar los programas de manera eficiente.

En este informe técnico, se estudian los conceptos básicos de la POO, como clase, objeto, atributo, método, encapsulamiento, herencia, polimorfismo y abstracción. También se exploran conceptos adicionales como interoperabilidad, escalabilidad y robustez.

Para desarrollar este informe, se utiliza el lenguaje de programación C++. Esto se debe a que C++ permite implementar de forma clara los principios fundamentales de la POO y es ampliamente utilizado en cursos de Estructura de Datos y Algoritmos. Se incluyen ejemplos cortos en C++ para facilitar la comprensión de cada concepto.

La investigación busca entender cómo la POO ayuda a diseñar programas más organizados, reutilizables y fáciles de mantener. También se analiza la importancia de seguir buenas prácticas de programación para desarrollar sistemas computacionales eficientes y robustos.

INDICE

I. INTRODUCCIÓN.....	4
1.1. Objetivos:.....	4
II. CONTENIDO DE INVESTIGACION.....	5
2.1 Programación Orientada a Objetos (POO) :.....	5
2.1.1 Clases :.....	6
2.1.2 Objetos :.....	7
2.1.3 Atributos :.....	8
2.1.4 Métodos :.....	9
2.1.5 Encapsulamiento :.....	10
2.1.6 Herencia :.....	11
2.1.7 Poliformismo :.....	12
2.1.8 Abstracción :.....	13
2.1.9 Interoperabilidad :.....	14
2.1.10 Escalabilidad :.....	15
2.1.11 Robustez :.....	16
III. DISCUCIONES.....	17
IV. CONCLUSIONES.....	18
V. RECOMENDACIONES.....	19
VI. REFERENCIAS BIBLIOGRAFICAS.....	20

I. INTRODUCCIÓN

La Programación Orientada a Objetos (POO) es uno de los métodos de programación más relevantes en la actualidad. Se utiliza mucho porque puede representar problemas del mundo real con objetos y clases. Esto hace que los programas sean más organizados, reutilizables y fáciles de mantener. Por eso, es muy importante en el desarrollo de software y en la implementación de estructuras de datos y algoritmos.

El lenguaje de programación C++ tiene características orientadas a objetos. Estas características ayudan a crear aplicaciones eficientes y estructuradas. Con C++, se pueden aplicar conceptos como encapsulamiento, herencia, polimorfismo y abstracción. Estos conceptos mejoran la calidad del código y optimizan el funcionamiento de los programas. Además, la POO hace que los sistemas sean más robustos, escalables e interoperables. Estas son características clave en el desarrollo de software moderno.

Este informe explica los principales conceptos de la Programación Orientada a Objetos. Incluye ejemplos básicos en C++ para ayudar a entender cada tema de manera práctica. También analiza la importancia de estos conceptos en el curso de Estructura de Datos y Algoritmos. Se enfoca en cómo se aplican para construir programas más eficientes y organizados.

1.1. Objetivos:

- Comprender los conceptos fundamentales de la Programación Orientada a Objetos en C++.
- Analizar la importancia de la POO en el desarrollo de estructuras de datos y algoritmos.

II. CONTENIDO DE INVESTIGACION

2.1 Programación Orientada a Objetos (POO) :

La Programación Orientada a Objetos, también llamada POO, es una forma de hacer programas. Esto se hace creando clases y objetos. De esta manera, se pueden representar cosas del mundo real dentro de un programa. Esto ayuda a que se pueda reutilizar código, mantener y hacer que las aplicaciones sean más grandes.

La POO se utiliza mucho hoy en día porque hace que los programas sean más organizados y fáciles de entender. En el lenguaje C++, la Programación Orientada a Objetos es algo fundamental. Se utiliza en muchos sistemas, videojuegos, aplicaciones de escritorio y software empresarial.

Características de la POO

- Uso de clases y objetos.
- Reutilización de código mediante herencia.
- Protección de datos con encapsulamiento.
- Flexibilidad mediante polimorfismo.
- Simplificación de sistemas complejos usando abstracción.
- Organización modular del código.
- Facilidad de mantenimiento y escalabilidad.

2.1.1 Clases :

Una clase es como un molde que define cómo serán los objetos. En esta clase, se pueden declarar características y acciones que todos los objetos tendrán.

Las clases son muy importantes en la programación orientada a objetos porque permiten representar cosas reales en un programa. Por ejemplo, en un sistema de la universidad, podrías tener clases como Estudiante, Curso o Profesor.

Usar clases hace que el software sea más organizado y fácil de mantener, porque toda la información relacionada está en un solo lugar.

Características de un objeto

- Define atributos y métodos.
- Permite crear objetos.
- Facilita la reutilización de código.
- Organiza mejor la información del programa.

Ejemplo:

```
class Auto {
public:
    std::string marca;
    std::string modelo;

    void mostrarInfo() {
        std::cout << "Marca: " << marca << std::endl;
        std::cout << "Modelo: " << modelo << std::endl;
    }
};
```

Contexto :

La clase Auto representa un vehículo mediante los atributos marca y modelo. Además, contiene el método mostrarInfo(), encargado de mostrar la información del objeto.

Este ejemplo demuestra cómo una clase agrupa datos y funciones relacionadas dentro de una sola estructura.

2.1.2 Objetos :

Un objeto es algo que se crea a partir de una clase. Esto significa que es una cosa específica que tiene sus propios datos y puede hacer cosas que se han definido en la clase.

Los objetos son útiles porque permiten trabajar con muchas cosas diferentes de manera independiente, aunque todas siguen el mismo patrón. Cada objeto tiene su propia información, incluso si pertenece al mismo grupo que otros objetos.

En la programación, los objetos son muy importantes porque ayudan a representar cosas del mundo real dentro de un sistema de computadora.

Características de un objeto:

- Se crea a partir de una clase.
- Posee identidad propia.
- Almacena datos independientes.
- Puede ejecutar métodos.

Ejemplo en C++ :

```
class Estudiante {  
public:  
    std::string nombre;  
  
    void mostrarNombre() {  
        std::cout << nombre << std::endl;  
    }  
};
```

Estudiante e1;

Contexto :

El objeto e1 fue creado utilizando la clase Estudiante. Este objeto puede almacenar información en el atributo nombre y utilizar el método mostrarNombre() para mostrar sus datos.

2.1.3 Atributos :

Un atributo es algo que se declara dentro de una clase. Sirve para guardar información sobre un objeto. Los atributos son como las características que tendrá cada objeto que creamos a partir de la clase.

Cada objeto tiene sus propios valores en estos atributos. Esto significa que objetos diferentes pueden tener información diferente, aunque sean de la misma clase.

Los atributos son útiles porque nos permiten representar cosas del mundo real dentro de un programa que utiliza objetos.

Características de un atributo :

- Almacena información del objeto.
- Se declara dentro de una clase.
- Puede tener distintos tipos de datos.
- Cada objeto posee sus propios valores

Ejemplo en C++ :

```
class Persona {  
public:  
    std::string nombre;  
    int edad;  
};
```

Contexto :

Los atributos **nombre** y **edad** almacenan información relacionada con cada objeto de tipo **Persona**.

2.1.4 Métodos :

Un método es básicamente una función que forma parte de una clase. Esto nos permite hacer cosas o seguir procesos relacionados con un objeto.

Los métodos muestran cómo se comportan los objetos en la Programación Orientada a Objetos.

Pueden hacer varias cosas, como mostrar información, cambiar atributos o realizar operaciones específicas según lo que el programa necesite hacer.

Características de un método :

- Define acciones de un objeto.
- Se declara dentro de una clase.
- Puede recibir parámetros.
- Puede retornar valores.

Ejemplo en C++ :

```
class Calculadora {  
public:  
    int sumar(int a, int b) {  
        return a + b;  
    }  
};
```

Contexto :

El método `sumar ( )` recibe dos números y devuelve el resultado de la suma.

2.1.5 Encapsulamiento :

El encapsulamiento es un principio importante en la Programación Orientada a Objetos. Esto significa que se protegen los datos internos de una clase. De esta manera, no se puede acceder directamente a ellos.

Para hacer esto, el lenguaje de programación C++ utiliza cosas llamadas modificadores de acceso. Estos son `private`, `public` y `protected`. El encapsulamiento es útil porque hace que el programa sea más seguro. También evita que se hagan cambios incorrectos en los datos del programa.

Características del encapsulamiento :

- Protege los datos internos.
- Controla el acceso a los atributos.
- Mejora la seguridad del programa.
- Facilita el mantenimiento del código.

Ejemplo en C++ :

```
class Cuenta {  
private:  
    double saldo;  
  
public:  
    void setSaldo(double s) {  
        saldo = s;  
    }  
  
    double getSaldo() {  
        return saldo;  
    }  
};
```

Contexto :

El atributo `saldo` se encuentra protegido mediante `private`, evitando el acceso directo desde fuera de la clase.

2.1.6 Herencia :

La herencia es un mecanismo que te permite crear clases nuevas basándote en otras que ya existen. La clase que se crea, llamada clase derivada, obtiene características y funciones de la clase original, conocida como clase base. Esto hace que sea más fácil reutilizar código.

Gracias a la herencia, puedes evitar repetir código en diferentes partes de un programa y organizar mejor tus proyectos grandes.

Características de la herencia :

- Reutiliza código existente.
- Permite crear jerarquías de clases.
- Facilita la extensión de funcionalidades.
- Mejora la organización del programa.

Ejemplo en C++ :

```
class Animal {  
public:  
    void sonido() {  
        std::cout << "Sonido general";  
    }  
};  
  
class Perro : public Animal {  
};
```

Contexto :

La clase `Perro` hereda el método `sonido()` de la clase `Animal`.

2.1.7 Poliformismo :

El polimorfismo nos permite que un mismo método se comporte de manera diferente dependiendo de la clase que lo está utilizando.

Esto hace que nuestro programa sea más flexible y nos permite trabajar con varios objetos de una forma más dinámica y sencilla.

Características del poliformismo :

- Un método puede comportarse de distintas formas.
- Mejora la flexibilidad del código.
- Facilita la reutilización de métodos.
- Permite trabajar con herencia.

Ejemplo en C++ :

```
class Animal {
public:
    virtual void sonido() {
        std::cout << "Sonido general";
    }
};

class Gato : public Animal {
public:
    void sonido() override {
        std::cout << "Miau";
    }
};
```

Contexto :

El método `sonido()` cambia su comportamiento en la clase `Gato`.

2.1.8 Abstracción :

La abstracción se trata de mostrar solo lo importante de un objeto y esconder los detalles que no son necesarios.

Este principio ayuda a hacer que los sistemas complejos sean más fáciles de entender y usar los objetos en un programa se vuelve más sencillo.

Características de la abstracción :

- Oculta detalles internos.
- Simplifica el uso del programa.
- Mejora la organización del código.
- Facilita el mantenimiento del software.

Ejemplo en C++ :

```
class Vehiculo {  
public:  
    void encender() {  
        std::cout << "Vehiculo encendido";  
    }  
};
```

Contexto :

El usuario puede utilizar el método `encender ( )` sin necesidad de conocer cómo funciona internamente.

2.1.9 Interoperabilidad :

La interoperabilidad es la capacidad de un sistema para comunicarse y compartir información con otros sistemas o aplicaciones.

En el desarrollo de software, la interoperabilidad es importante porque permite combinar programas hechos en diferentes lenguajes o plataformas.

Características de la interoperabilidad :

- Permite compartir información.
- Facilita la integración entre sistemas.
- Mejora la compatibilidad de software.
- Optimiza la comunicación de aplicaciones.

Ejemplo en C++ :

```
class BaseDatos {  
public:  
    void conectar() {  
        std::cout << "Conexion establecida";  
    }  
};
```

Contexto :

La clase representa una conexión que puede ser utilizada para comunicarse con otros sistemas o servicios.

2.1.10 Escalabilidad :

La escalabilidad es la capacidad de un sistema para crecer y adaptarse a nuevas necesidades sin afectar su funcionamiento principal.

La programación orientada a objetos facilita la escalabilidad porque utiliza clases que se pueden reutilizar y un código que está organizado de manera modular.

Características de la escalabilidad :

- Facilita ampliar sistemas.
- Reduce modificaciones complejas.
- Mejora el mantenimiento del software.
- Permite agregar nuevas funcionalidades.

Ejemplo en C++ :

```
class Usuario {  
public:  
    std::string nombre;  
};  
  
class Administrador : public Usuario {  
};
```

Contexto :

La clase `Administrador` amplía las funcionalidades de `Usuario` sin modificar la estructura original.

2.1.11 Robustez :

La robustez es la capacidad de un programa para funcionar correctamente aunque haya errores o entradas inválidas.

Un programa robusto es aquel que evita fallos inesperados y hace que el sistema sea más estable. Esto significa que el programa sigue funcionando bien incluso cuando algo sale mal.

Un software robusto es muy importante porque reduce los fallos inesperados y mejora la estabilidad del sistema. Esto nos permite trabajar de manera más eficiente y sin interrupciones.

Características de la robustez :

- Maneja errores correctamente.
- Reduce fallos del sistema.
- Mejora la estabilidad del software.
- Incrementa la seguridad del programa.

Ejemplo en C++ :

```
class Validacion {  
public:  
    bool verificar(int numero) {  
        return numero > 0;  
    }  
};
```

Contexto :

El método `verificar()` comprueba si el número ingresado es válido antes de continuar con el proceso.

III. DISCUCIONES

El estudio de la Programación Orientada a Objetos (POO) me hizo entender su importancia en el desarrollo de software. También vi cómo ayuda a organizar programas en el curso de Estructura de Datos y Algoritmos.

La Programación Orientada a Objetos (POO) facilita la creación de aplicaciones más estructuradas. Esto se logra mediante el uso de clases y objetos. Así puedo representar entidades reales dentro de un sistema computacional.

Durante la investigación, encontré que conceptos como clase, objeto, atributo y método son clave. Ayudan a construir programas organizados y reutilizables.

También aprendí sobre principios como encapsulamiento, herencia, polimorfismo y abstracción. Estos mejoran la seguridad, flexibilidad y mantenimiento del código.

La interoperabilidad, escalabilidad y robustez son características importantes en el desarrollo de software moderno. Permiten crear sistemas que se comunican con otras aplicaciones, crecen según sea necesario y manejan errores de manera segura.

Los ejemplos en C++ me ayudaron a comprender cada concepto de forma práctica. Demostraron la utilidad de la Programación Orientada a Objetos en la construcción de programas eficientes y bien estructurados.

IV. CONCLUSIONES

La Programación Orientada a Objetos es un enfoque clave en el desarrollo de software actual. Ayuda a crear programas más organizados, reutilizables y fáciles de mantener.

Al estudiar conceptos como clases, objetos, atributos y métodos, se entiende mejor cómo funciona la POO en C++.

Principios como encapsulamiento, herencia, polimorfismo y abstracción son fundamentales. Mejorarán la seguridad, flexibilidad y escalabilidad del software.

Estos principios permiten crear sistemas más robustos y adaptables a diferentes necesidades en programación.

Desarrollar ejemplos prácticos en C++ ayudó a reforzar los conocimientos teóricos. Se destaca la importancia de la Programación Orientada a Objetos en el curso de Estructura de Datos y Algoritmos.

Esto muestra su aplicación en la creación de programas eficientes y bien estructurados.

V. RECOMENDACIONES

- Practicar constantemente ejercicios de Programación Orientada a Objetos en C++ para fortalecer la comprensión de clases, objetos y métodos.
- Aplicar principios como encapsulamiento, herencia y polimorfismo en programas pequeños antes de desarrollar proyectos más complejos.
- Mantener el código organizado y modular para facilitar el mantenimiento y la reutilización de componentes.
- Implementar validaciones y manejo de errores con el fin de desarrollar programas más robustos y seguros.
- Continuar investigando nuevas técnicas y buenas prácticas relacionadas con la Programación Orientada a Objetos y estructuras de datos.

VI. REFERENCIAS BIBLIOGRAFICAS

- Curso de C++ desde CERO para PRINCIPIANTES (Completo)
<https://www.youtube.com/watch?v=jS6wb263CIM&t=33192s>
- Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ primer* (5th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2005). *Data structures and algorithm analysis in C++* (3rd ed.). Pearson Addison-Wesley.